АВРОРА
СВОЯ СИСТЕМА

Интенсив Flutter разработки под ОС Аврора

День 3 - Основы разработки плагинов

Овчинников Сергей
Инженер-разработчик ОМП

Содержание мастер класса

- Классификация плагинов
- Разработка Platform плагинов
- Портирование плагинов
- Особенности разработки плагинов для ОС Аврора

Классификация плагинов

Классификация плагинов

Типы плагинов:

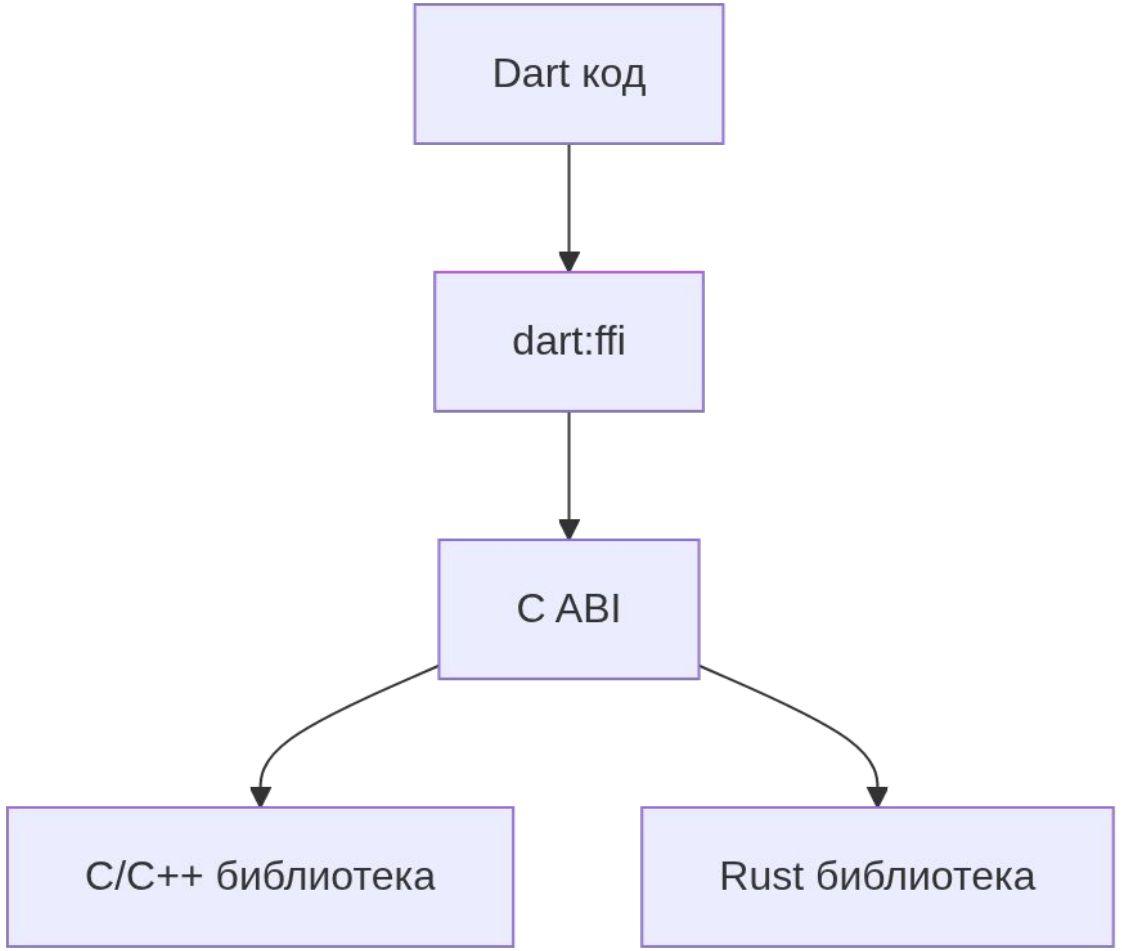
- FFI плагины
- Dbus плагины
- Platform channel плагины
- Federated плагины

Классификация плагинов

FFI плагины

Основа: dart::ffi

Генератор: ffigen



Классификация плагинов

FFI плагины

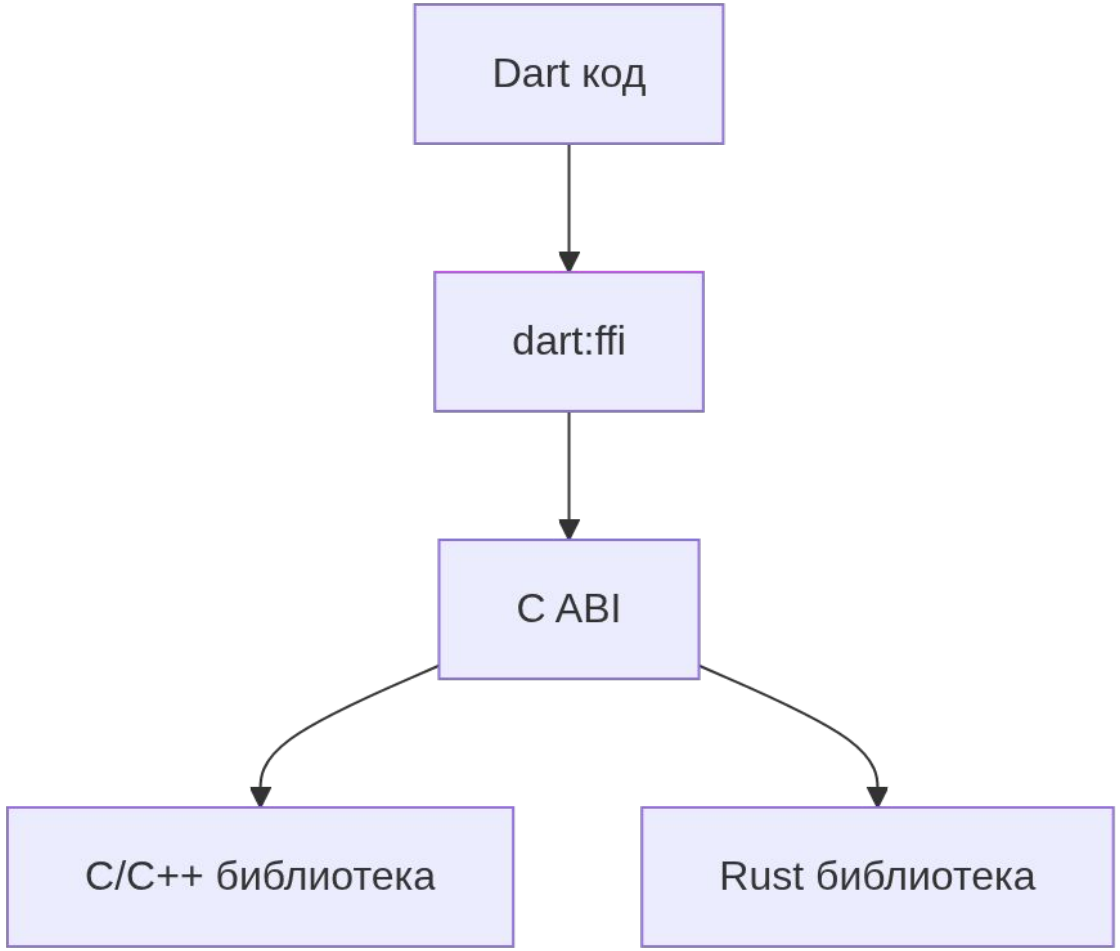
Плюсы:

- Прямой вызов из нативного кода
- Единый интерфейс C
- Прямая работа с памятью
- Высокая производительность

Минусы:

- Ограничение типов языка C
- Высокая сложность разработки
- Сборка под каждую платформу

Примеры: pdfrx_aurora

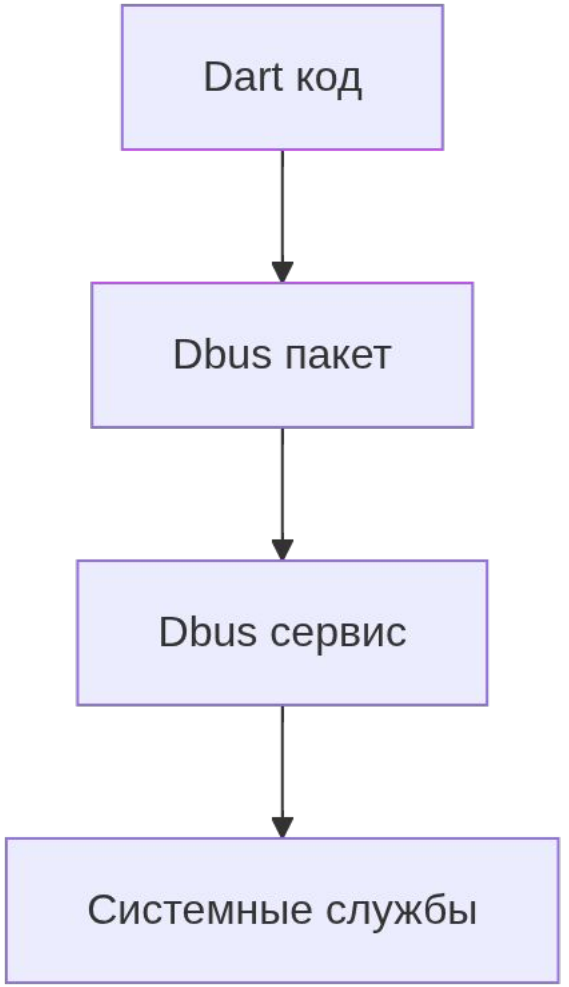


Классификация плагинов

Dbus плагины (пакеты)

Сервис dbus

Dart пакет dbus



Классификация плагинов

Dbus плагины (пакеты)

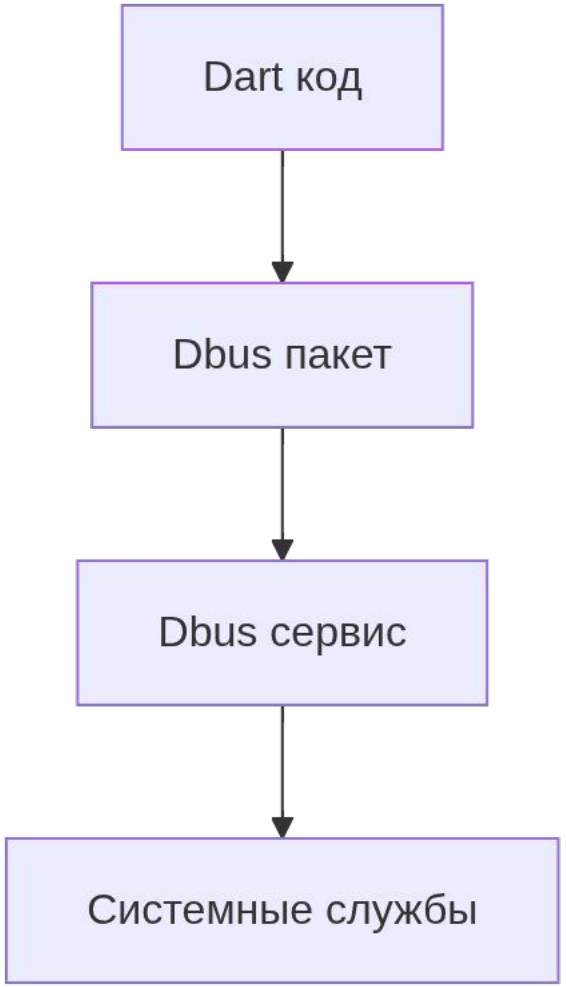
Плюсы:

- Отсутствие нативного кода
- Производительность

Минусы:

- Ограничение наличия Api
- Сложная отладка
- Ручная типизация

Примеры: geoclue_aurora

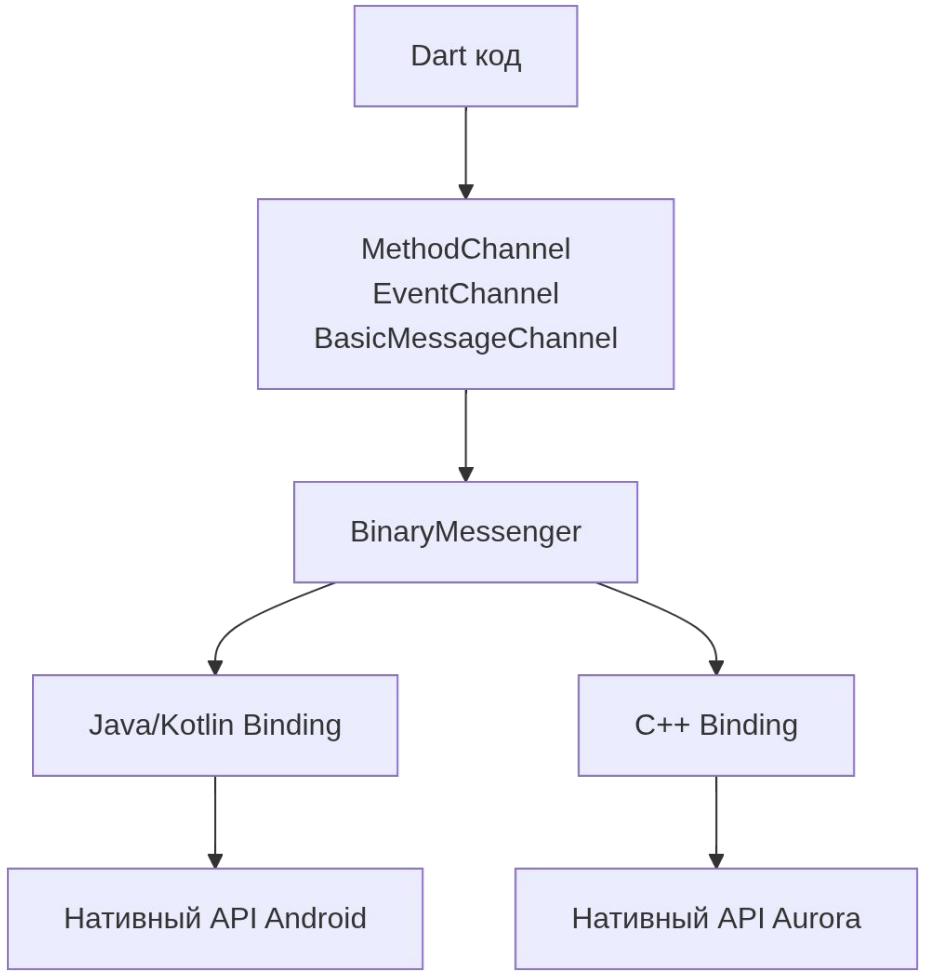


Классификация плагинов

Platform channel плагины

Классический подход реализации плагинов

Документация



Классификация плагинов

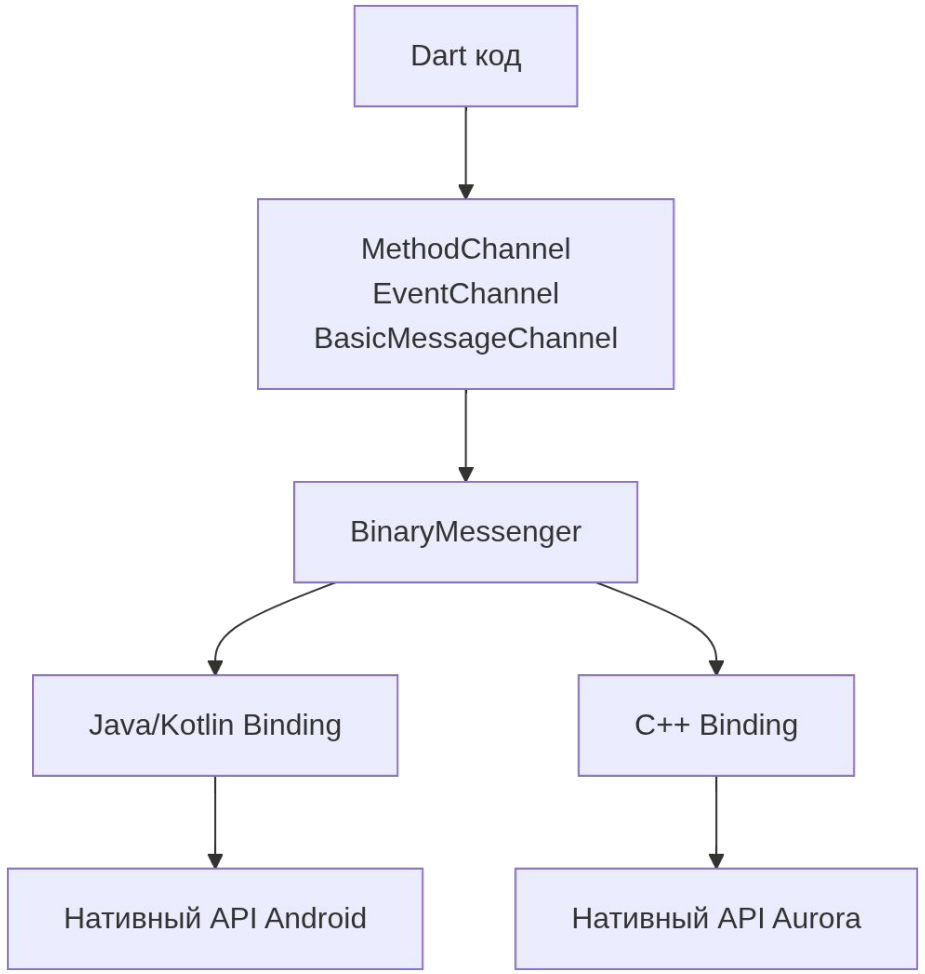
Platform channel плагины

Плюсы:

- Единое api для платформ
- Статическая типизация
- Баланс производительности и удобства

Минусы:

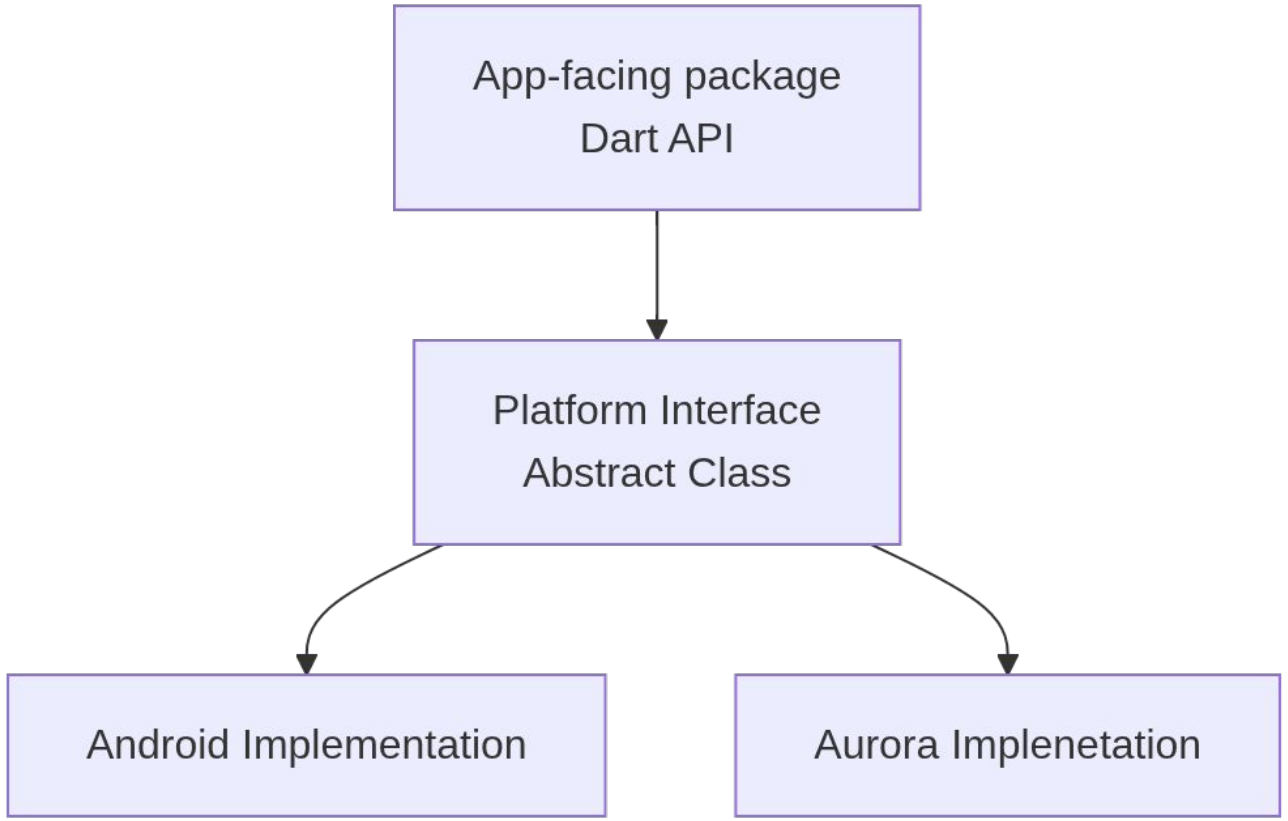
- Накладные расходы сериализацию/десериализацию
- Ручное преобразование типов и обработка ошибок



Классификация плагинов

Federated плагины

Документация



Классификация плагинов

Federated плагины

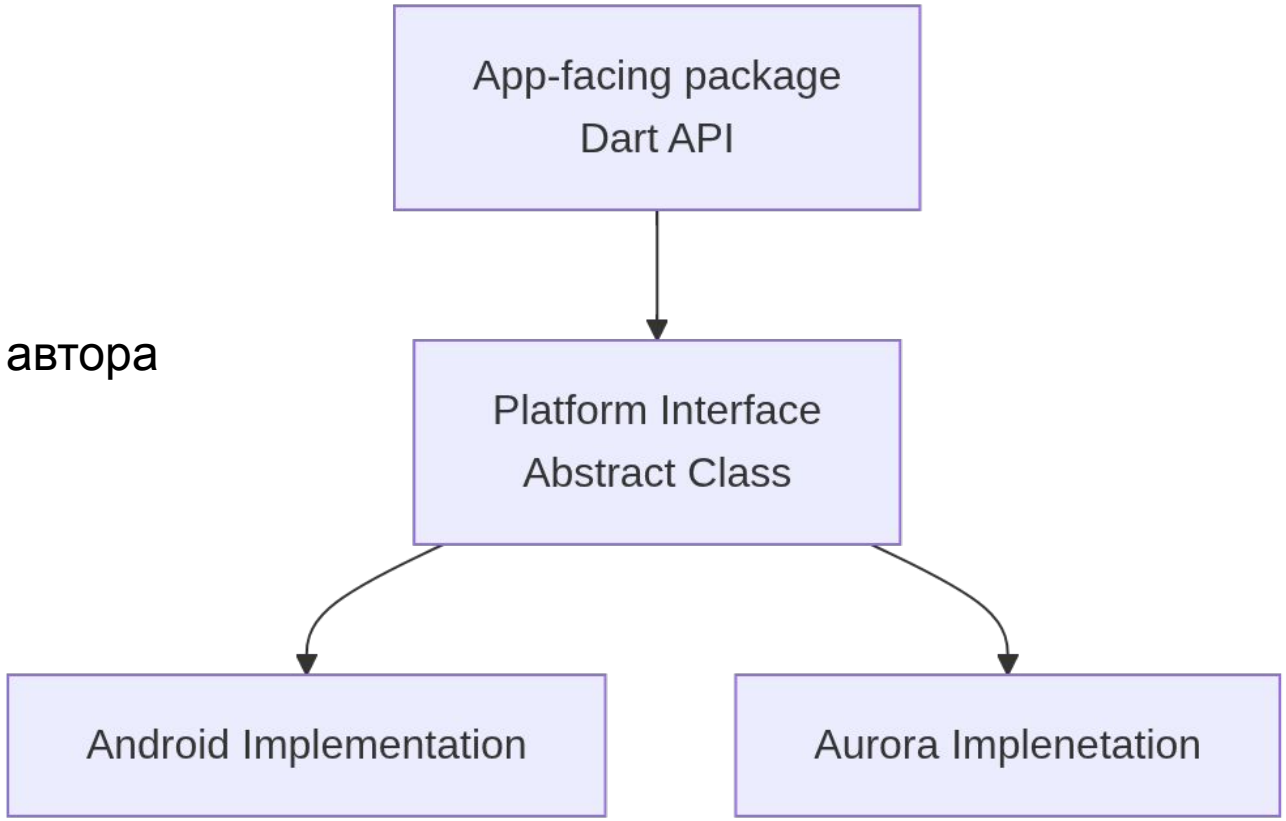
Плюсы:

- Независимые части
- Раздельные реализации
- Загрузка необходимой части
- Добавление реализации без одобрения автора

Минусы:

- Усложненная разработка
- Усложненное версионирование

Примеры: camera_aurora, vibration_aurora, url_launcher_aurora



Разработка Platform плагинов

Разработка Platform плагинов

Генерация шаблона плагина

Шаблоны:

- plugin - platform channel плагин
- plugin_ffi - ffi плагин

```
flutter-aurora create \
    --platforms=aurora \
    --template=plugin \
    --org=ru.aurora\
    plugin_demo
```

Разработка Platform плагинов

Подходы к созданию Platform channel плагинов

	flutter::Plugin	**flutter::BasicPlugin**
Плюсы	• Контроль над результатами • Работа с асинхронностью	• Простота создания • Сразу реализован method channel
Минусы	• Сложная реализация • Неочевидные ошибки	• Нет возможности использовать асинхронность • Ограничение в каналах
Применение	• Продвинутые плагины	• Простые плагины

Разработка Platform плагинов

Пример реализации на основе `flutter::BasicPlugin`

```
class PLUGIN_EXPORT PluginDemoPlugin final : public flutter::BasicPlugin<> {

public:
    explicit PluginDemoPlugin(flutter::PluginRegistrar* registrar);
    Result HandleMethodCall(const std::string& name, const EncodableValue* arguments)
final;
private:
    EncodableValue onGetPlatformVersion();
};
```

Разработка Platform плагинов

Пример реализации на основе `flutter::BasicPlugin`

```
PluginDemoPlugin::PluginDemoPlugin(flutter::PluginRegistrar*
registrar):flutter::BasicPlugin<>("plugin_demo", registrar->messenger()) {}

PluginDemoPlugin::Result PluginDemoPlugin::HandleMethodCall(const std::string& name,[[maybe_ununsed]]
const flutter::EncodableValue* arguments) {
    if (name == "getPlatformVersion") {
        return onGetPlatformVersion();
    }
    return Result(flutter::NotImplemented()
);
}

EncodableValue PluginDemoPlugin::onGetPlatformVersion() {
    return EncodableValue("Aurora OS.");
}
```

Разработка Platform плагинов

Пример реализации на основе `flutter::Plugin`

```
class PLUGIN_EXPORT PluginDemoPlugin final : public flutter::Plugin {

public:
    explicit PluginDemoPlugin(std::unique_ptr<MethodChannel>
methodChannel);

private:
    flutter::EncodableValue onGetPlatformVersion();
    std::unique_ptr<MethodChannel> m_methodChannel;
};
```

Разработка Platform плагинов

Пример реализации на основе `flutter::Plugin`

```
PluginDemoPlugin::PluginDemoPlugin(std::unique_ptr<flutter::MethodChannel<flutter::EncodableValue>>
methodChannel):
m_methodChannel(std::move(methodChannel)) {
    m_methodChannel->SetMethodCallHandler([this](const MethodCall& call, std::unique_ptr<MethodResult>
result) {
        const auto name = call.method_name();
        if (name == Methods::PlatformVersion) {
            result->Success(onGetPlatformVersion());
        } else {
            result->NotImplemented();
        }
    });
}

flutter::EncodableValue PluginDemoPlugin::onGetPlatformVersion() {
    return flutter::EncodableValue("Aurora OS.");
}
```

Портирование плагинов

Портирование плагинов

Портирование плагинов

1. Анализ pubspec.yaml
2. Анализ исходного кода
3. Анализ возможностей ОС
4. Анализ сторонних библиотек

Портирование плагинов

Варианты реализации плагина под платформу

- Реализовать интерфейс и нативную часть
- Реализовать нативную часть
- Использовать ffi библиотеку
- Сделать обертку
- Использовать dbus сервис
- Комбинировать подходы

Разработка Platform плагинов

Особенности разработки плагинов для ОС Аврора

Взаимодействие с QT

- Аврора ОС использует Qt 5.6;
- Один из главных фреймворков разработки для ОС Аврора;
- Глубокая интеграция с ОС и оборудованием;
- Реализован механизм сигналов и слотов для Flutter;

Особенности разработки плагинов для ОС Аврора

Подключение aurora_client_wrapper

Подключение библиотек:

```
# Add client_wrapper source
include(AuroraClientWrapper)
ADD_CLIENT_WRAPPER_SOURCE(${PLUGIN_NAME})
```

Подключение плагина:

```
aurora_client_wrapper: ^1.0.3
```

Особенности разработки плагинов для ОС Аврора

Подключение Qt

Подключение библиотек:

```
find_package(Qt5 COMPONENTS Core
  REQUIRED)

target_link_libraries(${PLUGIN_NAME
} PUBLIC Qt5::Core)
```

Подключение QObject:

```
add_library(${PLUGIN_NAME} SHARED
    include/${PROJECT_NAME}/my_class.h
    my_class.cpp
)

set_target_properties(${PLUGIN_NAME}
PROPERTIES AUTOMOC ON)
```

Особенности разработки плагинов для ОС Аврора

Включение сигналов и слотов

pubspec.yaml

```yaml
flutter:
  plugin:
    platforms:
      aurora:
        useQt: true
```

Особенности разработки плагинов для ОС Аврора

Desktop файл



Документация

```
Type=Application
X-Nemo-Application-Type=silica-qt5
Icon={название_пакета}
Exec={команда_запуска}
Name={отображаемое_название_приложения_на_английском_языке}
Name[ru]={отображаемое_название_приложения_на_русском_языке}

[X-Application]
Permissions={список_разрешений}
OrganizationName={доменное_имя}
ApplicationName={название_приложения}
ExecDBus={команда_запуска_для_службы_D-Bus}
ExportDBusInterfaces={название_интерфейса_1};{название_интерфейса_2};...
```

Особенности разработки плагинов для ОС Аврора

Spec-файл



Документация

```
%global __requires_exclude
  ^lib(dconf|flutter-embedder|maliit-glib|.+_platform_plugin)\\.so.*$

Name: ru.aurora.my_app
Summary: My application
Version: 0.1.0
Release: 1

BuildRequires: cmake
BuildRequires: ninja
```

Особенности разработки плагинов для ОС Аврора

Автоматизация сборки и исключения зависимостей:

pubspec.yaml:

```yaml
flutter:
  plugin:
    implements: vibration
    platforms:
      aurora:
        requiresExclude: ['native_dep_1', 'native_dep_2']
        buildRequires:
          - pkgconfig(Qt5Core)
          - pkgconfig(Qt5Feedback)
```

Особенности разработки плагинов для ОС Аврора

Конфигурирование:

pubspec.yaml:

```yaml
flutter:
  plugin:
    implements: vibration
    platforms:
      aurora:
        dartPluginClass: PluginName
        pluginEntryPointHeader: plugin.h
        pluginEntryPoint:
          PluginNamePluginRegisterWithRegistrar
```

Особенности разработки плагинов для ОС Аврора

Отладка нативного C++ кода

Способы:



printf
aurora_spdlog



gdb,
flutter cli

Дополнительная информация:

- Системный журнал devel-su systemctl
- Запуск с флагом композитора WAYLAND_DEBUG=1
- Монитор dbus: dbus-monitor

Спасибо за внимание!